

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010

MLOPs Report

Group ID: 25-26J-148

Project Title: Automatic Cricket Video Segmentation By Player Identity And Event Type

1. IT22141606: Automatic Cricket Video Segmentation By Player Identity And Event Type

		IT22141606
Data Pipeline	Data Sources:	The data used for this research consists of publicly available cricket match videos, mainly collected from YouTube. These videos contain cricket match footage, scoreboard overlays, player names, umpire signals, and match events. The system uses these videos as input to identify player-related events such as fours, sixes, wickets, and no-action moments.
	Data Preprocessing:	The uploaded cricket video is preprocessed before player identification, event detection, and retrieval. First, the video is read using OpenCV and frames are extracted at selected time intervals to reduce unnecessary processing. For scoreboard/player identification, - The lower scoreboard region of each selected frame is cropped because player names and scores usually appear in that area. EasyOCR is then applied to extract text from the scoreboard. The extracted OCR text is cleaned by converting it to uppercase, removing unwanted symbols, correcting common OCR errors, filtering non-name words, and removing score-like numbers. Player name candidates are then identified using cleaned OCR text, NER-based player recognition, fuzzy matching, and team roster comparison. For event identification, - selected video frames are converted into image format, resized to 224 × 224 pixels, transformed into tensors, and normalized using ImageNet mean and standard deviation values. The preprocessed frame is first passed to the umpire detection

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010

MLOPs Report

		model to check whether an umpire is present. If an umpire is detected, the frame is then passed to the umpire pose/event classification model to identify events such as FOUR, SIX, WICKET/OUT, or NO-ACTION. During preprocessing, timestamps are also generated for each frame so that detected events and identified players can be linked to the correct video time. This complete preprocessing pipeline prepares the video data for OCR, player identification, event classification, player-event association, and dashboard-based retrieval.
	Data storage and versioning	The system stores uploaded videos, processed match folders, generated video segments, extracted OCR results, event metadata, player-event records, and timestamp information inside structured backend project directories. Match-related data is stored in JSON/CSV-style structured formats so that it can be retrieved by the dashboard. Model files such as the umpire detection model, umpire pose classification model, and player NER model are stored in separate model directories. Versioning can be maintained by organizing datasets, model checkpoints, and processed outputs using clear folder names, model names, and experiment records. (Cloud deployment was not used because cricket videos and generated segments require large storage, bandwidth, and server resources. Therefore, the system was tested on localhost as a practical and cost-effective approach for the current research scope.)
Model Development	Model Selection	The project uses a multimodal approach. - EfficientNet-B0 is selected for umpire detection because it is lightweight and suitable for image classification tasks. - ResNet-18 is selected for umpire pose/event classification because it is a reliable CNN architecture for classifying visual event patterns. - EasyOCR is used for scoreboard text extraction, while a transformer-based NER model is used to identify player names and roles from extracted text. Fuzzy matching is also used to improve player name matching with team rosters.
	Model Training	The umpire detection model and umpire pose classification model are trained using labelled image datasets. The umpire detection model classifies whether an umpire is present or not, while the pose classification model identifies event-related umpire signals such as FOUR, SIX, WICKET/OUT, and NO-ACTION. The player NER model is trained or fine-tuned to

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010

MLOPs Report

		recognize player-related entities from scoreboard text. After training, the best model checkpoints are saved and loaded into the backend for inference.
Model Integration	Tools used	- The backend is developed using Python and FastAPI. OpenCV is used for video processing and frame extraction. - EasyOCR is used for scoreboard OCR. PyTorch, torchvision, and timm are used for deep learning model loading and inference. - EfficientNet-B0 and ResNet-18 are used for event-related visual classification. - Hugging Face Transformers is used for the player NER model. The frontend dashboard is developed using React, where users can view matches, search events, filter players, and play detected video segments.
Model Deployment	Testing Environments	The system is tested in a local development environment using the FastAPI backend and React frontend. Cricket videos are uploaded through the system and processed to check OCR extraction, player identification, event detection, timestamp synchronization, and dashboard retrieval. The system is tested using different match videos to observe how video quality, scoreboard clarity, umpire visibility, and player name formats affect the final results.
	Deployment Platform	The system is deployed and tested in a local development environment using localhost. The backend runs using FastAPI, and the frontend runs as a React web application. The processed videos and match data are served through the backend, while the dashboard provides the user interface for searching and viewing cricket events. Cloud deployment was not used in the current research scope because cricket videos are large in size and the system also generates processed video segments and match metadata. Hosting these files online would require additional cloud storage, bandwidth, and server resources. Therefore, localhost deployment was selected as a practical and cost-effective approach for development, testing, and academic demonstration. In future, the system can be deployed on a cloud server with sufficient storage, authentication, and processing support.
	Deployment Method	The system is deployed and tested using a localhost-based development setup. The trained model checkpoints are saved inside the backend model folders and loaded during application runtime. When a user uploads a cricket video through the frontend, the FastAPI backend receives the video and processes it using the complete pipeline. The backend extracts frames, applies OCR for scoreboard text, performs player identification, detects

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010

MLOPs Report

		umpire-signal events using the trained models, generates timestamps, and stores the processed event-player records and video segment details. The React frontend dashboard communicates with the backend APIs to retrieve match details, detected events, player information, timestamps, and playable video segments. This creates an end-to-end workflow from video upload to event-based retrieval. Cloud deployment was not used in the current stage because cricket videos and generated segments require large storage capacity, bandwidth, and server resources. Therefore, localhost deployment was used as a practical and cost-effective method for development, testing, and academic demonstration.
Future Enhancements	Model Improvement:	Future improvements can include increasing the training dataset size, using more diverse cricket videos, improving OCR accuracy for different scoreboard layouts, adding more event classes such as wide, no-ball, review, and run-out, and improving player-event association accuracy. The system can also include model monitoring, automatic error logging, confidence score display, manual correction options, and retraining with corrected results. In future deployment, stronger cloud-based MLOps practices such as dataset versioning, model versioning, CI/CD pipelines, monitoring dashboards, and automated retraining can be added.